

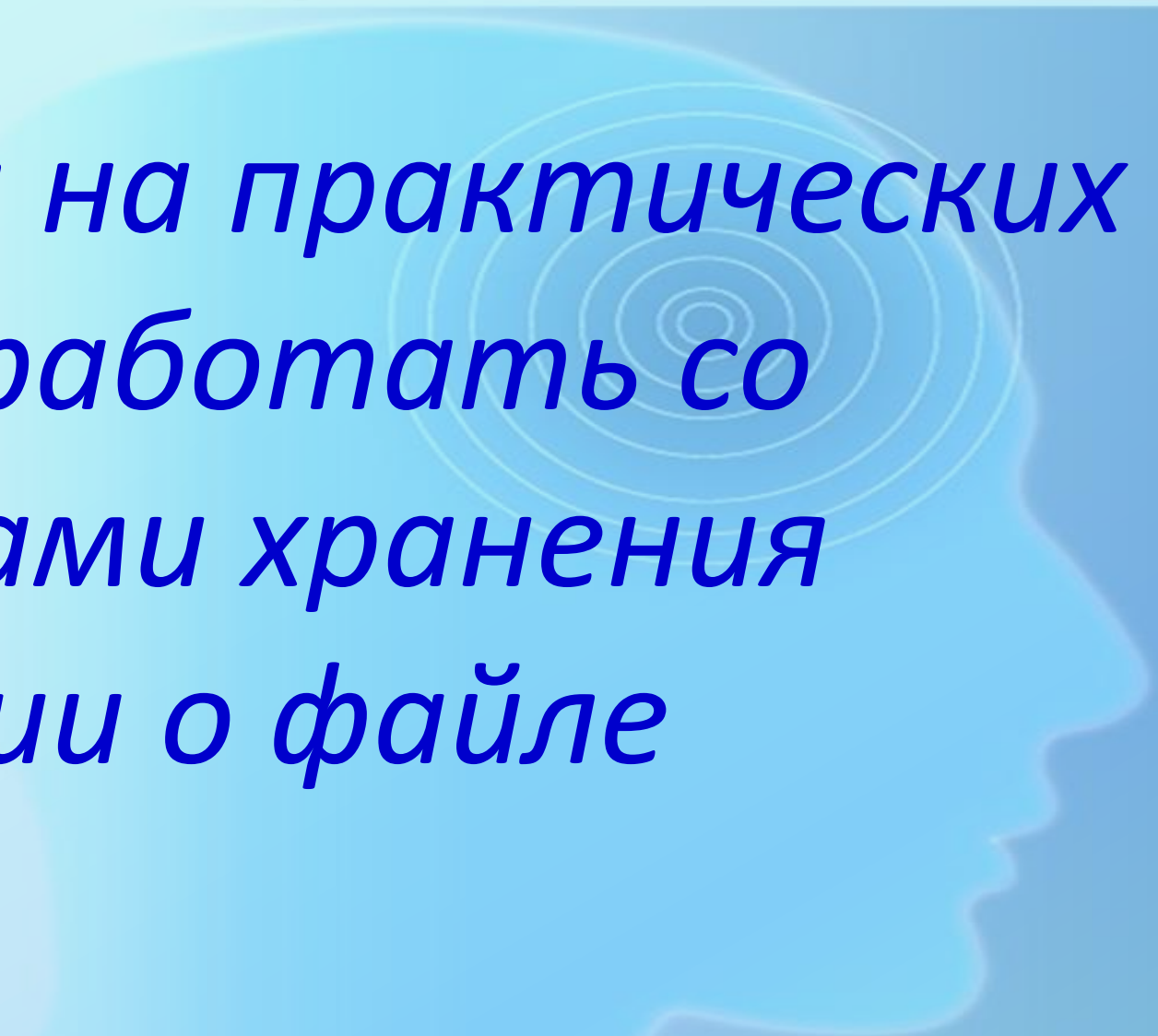


тема

Структуры хранения информации о файле

Цель

Научиться на практических примерах работать со структурами хранения информации о файле



Задачи

Научиться работать со структурами:

- FILE,
- struct stat,
- файловые дескрипторы,
- буферы и связанными с ними механизмы

Часть 1.

Структура FILE (примеры 1–7)

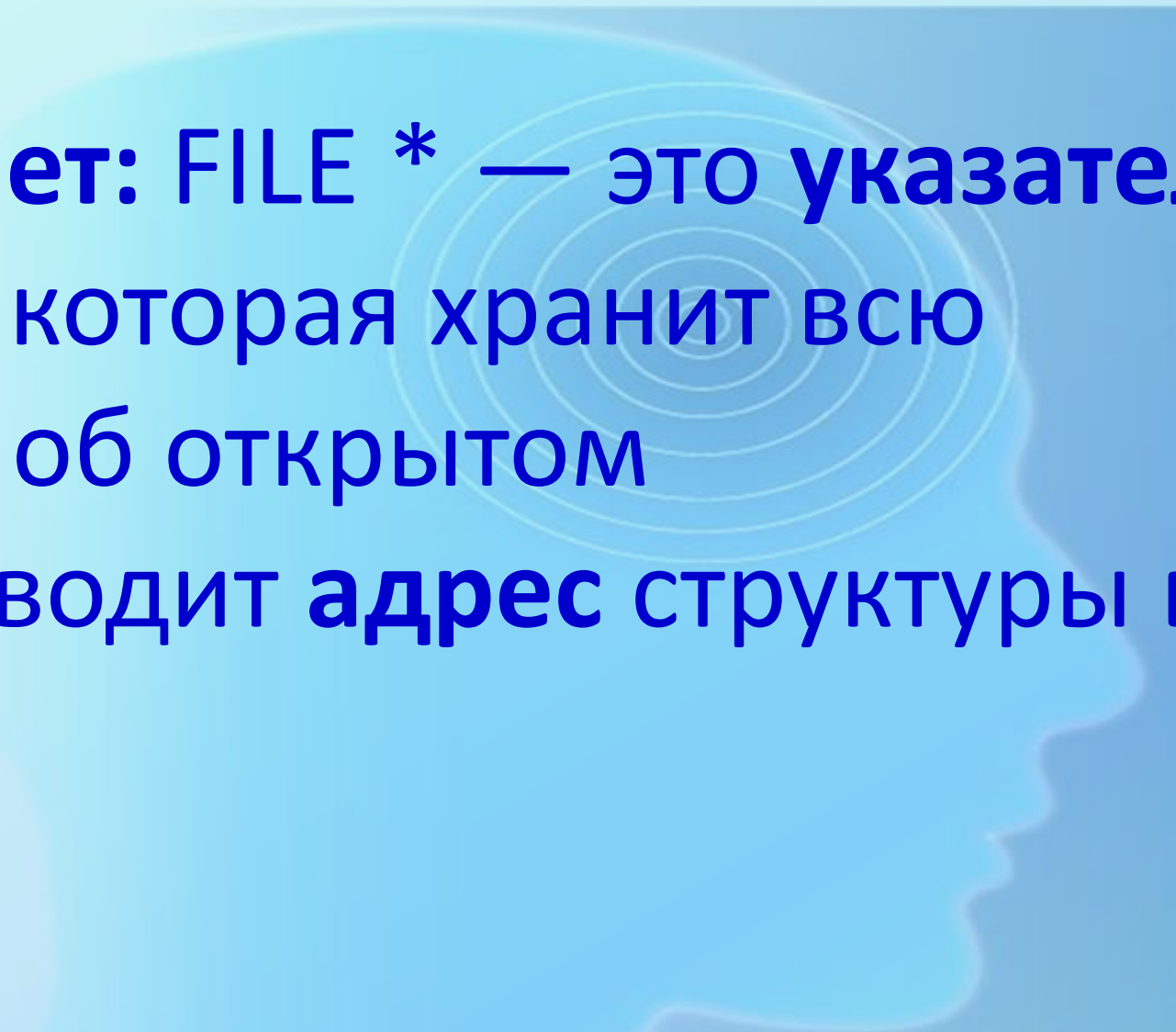


Пример 1. Что такое FILE * — объявление и проверка

```
#include <stdio.h>
```

```
int main(void) {  
    FILE *fp = fopen("hello.txt", "r");  
    if (fp == NULL) {  
        perror("fopen");  
        return 1;  
    }  
    printf("Файл открыт, указатель: %p\n", (void *)fp);  
    fclose(fp);  
    return 0;  
}
```





Что показывает: FILE * — это указатель на структуру, которая хранит всю информацию об открытом файле. %p выводит адрес структуры в памяти.

Подключение библиотеки

```
#include <stdio.h>
```

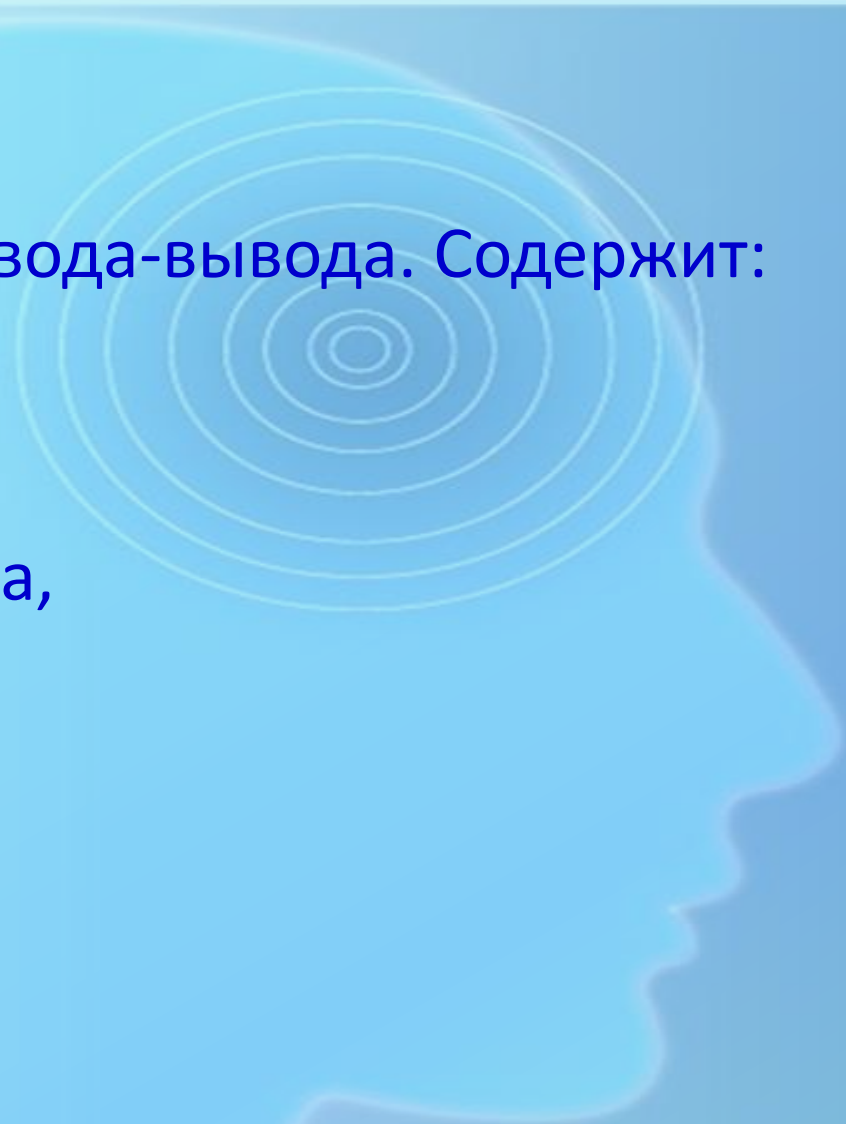
stdio.h — стандартная библиотека ввода-вывода. Содержит:

FILE — тип для работы с файлами,

fopen, fclose — открытие/заккрытие,

fgetc, fputc — чтение/запись символа,

EOF — признак конца файла.



Открытие исходного файла

- `FILE *in = fopen("in.txt", "r");`
- `fopen("in.txt", "r")` — открывает файл `in.txt` **только для чтения**.
- Режим `"r"`: файл **должен существовать**, иначе `fopen` вернёт `NULL`.
- `FILE *in` — указатель на структуру `FILE` — «дескриптор» файла.

Открытие файла для записи

```
FILE *out = fopen("out.txt", "w");
```

Режим "w": файл открывается **для записи**.

Если файла **нет** — он **создаётся**.

Если файл **есть** — его
содержимое **стирается** (перезаписывается).

Пример 2. Размер структуры FILE

```
#include <stdio.h>
```

```
int main(void) {  
    printf("sizeof(FILE) = %zu байт\n", sizeof(FILE));  
    return 0;  
}
```

Что показывает: структура FILE занимает **фиксированный объём памяти**. На разных системах — **разный**, но обычно **от 100 до 500 байт**.

Пример 3. Позиция в файле через ftell

```
#include <stdio.h>

int main(void) {
    FILE *fp = fopen("hello.txt", "r");
    if (!fp) return 1;

    int c;
    while ((c = fgetc(fp)) != EOF) {
        long pos = ftell(fp);
        printf("Символ '%c' на позиции %ld\n", c, pos);
    }

    fclose(fp);
    return 0;
}
```

Что показывает: внутри FILE хранится **текущая позиция** в файле. ftell её возвращает.

Пример 4. Флаги состояния: feof и ferror

```
#include <stdio.h>

int main(void) {
    FILE *fp = fopen("hello.txt", "r");
    if (!fp) return 1;

    while (fgetc(fp) != EOF) { }

    if (feof(fp)) printf("Достигнут конец файла\n");
    if (ferror(fp)) printf("Произошла ошибка\n");

    fclose(fp);
    return 0;
}
```

Что показывает: в FILE есть **флаги состояния** — EOF и error. Их проверяют через feof и ferror.

Пример 5. Сброс буфера через fflush

```
#include <stdio.h>

int main(void) {
    FILE *fp = fopen("out.txt", "w");
    if (!fp) return 1;

    fputs("Данные в буфере", fp);
    fflush(fp); // сброс буфера на диск
    printf("Буфер сброшен\n");

    fclose(fp);
    return 0;
}
```

Что показывает: FILE содержит **буфер**. fflush принудительно записывает его содержимое на диск.

Пример 6. Буферизация: setvbuf

```
#include <stdio.h>

int main(void) {
    FILE *fp = fopen("out.txt", "w");
    if (!fp) return 1;

    char buf[1024];
    setvbuf(fp, buf, _IOFBF, sizeof(buf)); // полная буферизация

    fputs("Данные", fp);
    fclose(fp);
    return 0;
}
```

Что показывает: можно **управлять буфером** FILE через setvbuf. Режимы:

_IOFBF — полная буферизация.

_IOLBF — построчная.

_IONBF — без буферизации.

Пример 7. Временный файл через tmpfile

```
#include <stdio.h>

int main(void) {
    FILE *fp = tmpfile();
    if (!fp) return 1;

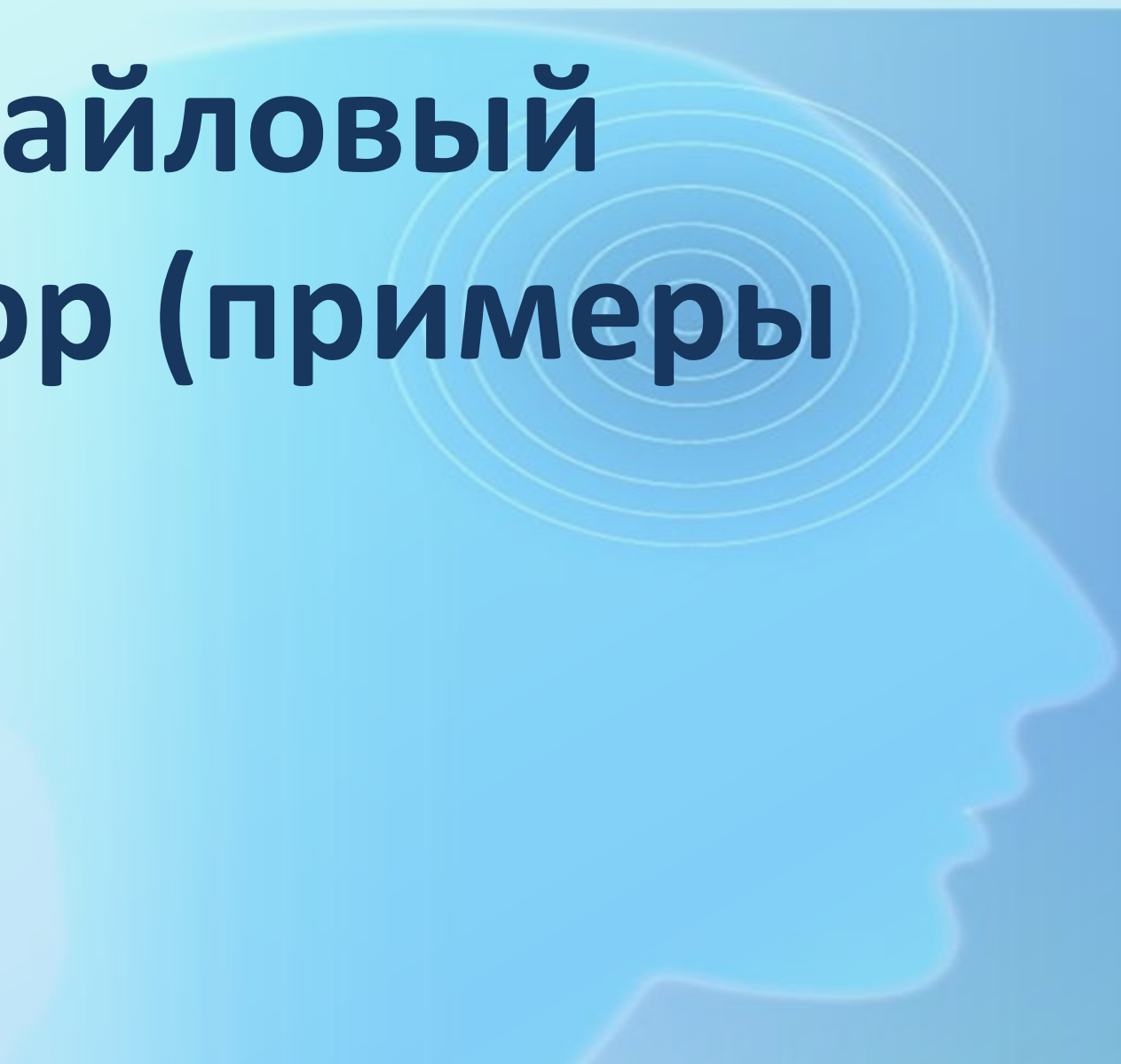
    fputs("Временные данные", fp);
    rewind(fp);

    char buf[100];
    fgets(buf, sizeof(buf), fp);
    printf("Прочитано: %s\n", buf);

    fclose(fp); // файл автоматически удаляется
    return 0;
}
```

Что показывает: tmpfile создаёт **временный файл**, который автоматически удаляется при закрытии. Структура FILE хранит его дескриптор.

Часть 2. Файловый дескриптор (примеры 8–12)



Пример 8. Получение дескриптора из FILE через fileno

```
#include <stdio.h>

int main(void) {
    FILE *fp = fopen("hello.txt", "r");
    if (!fp) return 1;

    int fd = fileno(fp);
    printf("Файловый дескриптор: %d\n", fd);

    fclose(fp);
    return 0;
}
```

Что показывает: внутри FILE хранится **файловый дескриптор** (int). fileno его возвращает.

Пример 9. Стандартные дескрипторы

```
#include <stdio.h>
```

```
int main(void) {  
    printf("stdin: fd = %d\n", fileno(stdin));  
    printf("stdout: fd = %d\n", fileno(stdout));  
    printf("stderr: fd = %d\n", fileno(stderr));  
    return 0;  
}
```

Что показывает: по умолчанию открыты **три дескриптора**:

- 0 — stdin,
- 1 — stdout,
- 2 — stderr.

Пример 10. Дескриптор через open (системный вызов)

```
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>

int main(void) {
    int fd = open("hello.txt", O_RDONLY);
    if (fd == -1) {
        perror("open");
        return 1;
    }
    printf("Дескриптор: %d\n", fd);
    close(fd);
    return 0;
}
```

Что показывает: системный вызов `open` возвращает **дескриптор** — целое число, индекс в таблице открытых файлов процесса.

Пример 11. Обёртка FILE вокруг дескриптора через fdopen

```
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>

int main(void) {
    int fd = open("hello.txt", O_RDONLY);
    if (fd == -1) return 1;

    FILE *fp = fdopen(fd, "r");
    if (!fp) return 1;

    int c;
    while ((c = fgetc(fp)) != EOF) putchar(c);

    fclose(fp); // закрывает и fp, и fd
    return 0;
}
```

Что показывает: можно **обернуть** низкоуровневый дескриптор в высокоуровневую структуру FILE через fdopen.

Пример 12. Дублирование дескриптора через dup

```
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>

int main(void) {
    int fd1 = open("hello.txt", O_RDONLY);
    if (fd1 == -1) return 1;

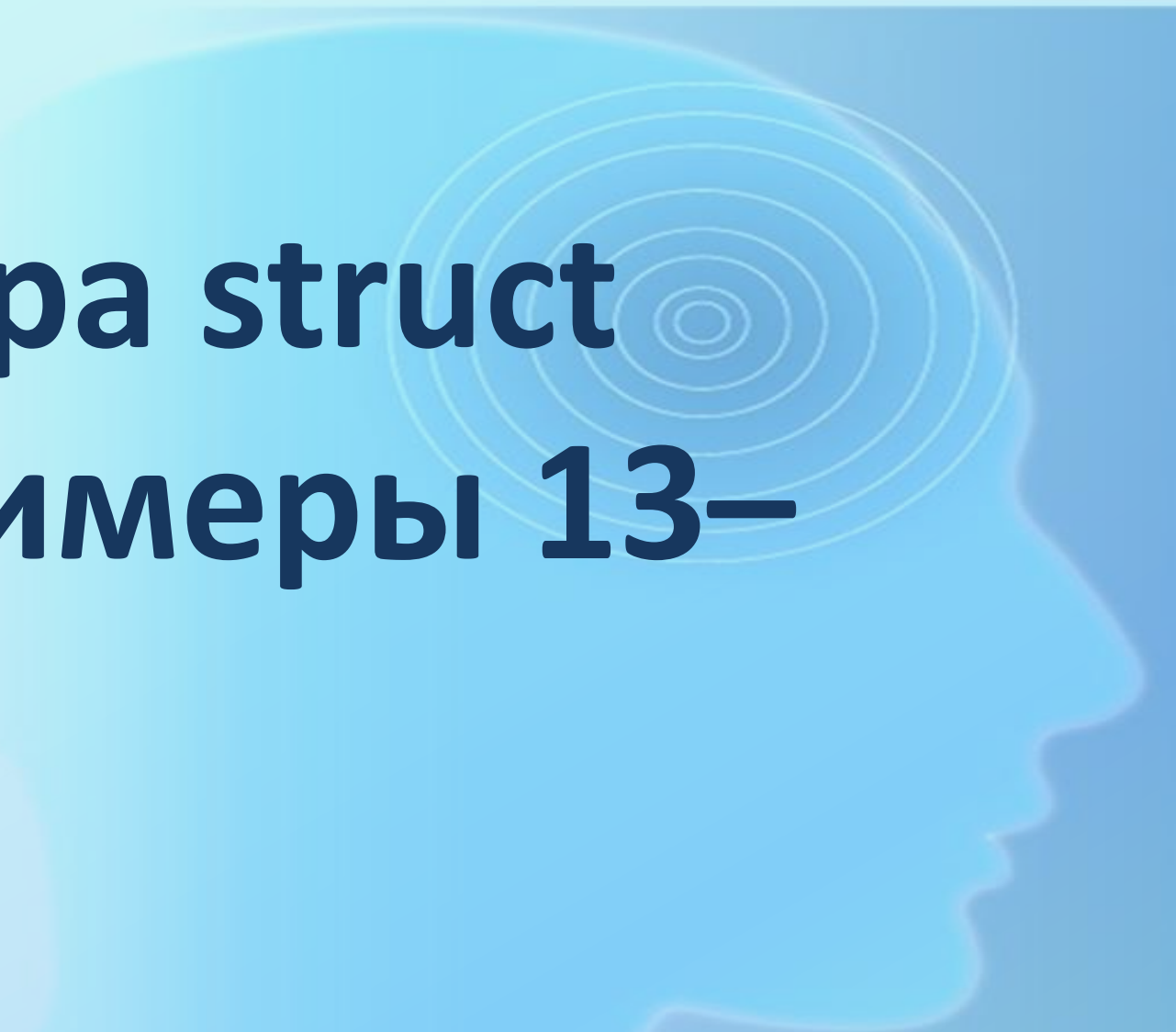
    int fd2 = dup(fd1);
    printf("fd1 = %d, fd2 = %d\n", fd1, fd2);

    close(fd1);
    close(fd2);
    return 0;
}
```

Что показывает: dup создаёт **копию дескриптора**, указывающую на тот же файл. Оба дескриптора независимы, но разделяют позицию.

Часть 3.

**Структура `struct`
`stat` (примеры 13–
17)**



Пример 13. Получение метаданных через stat

```
#include <stdio.h>
#include <sys/stat.h>

int main(void) {
    struct stat st;
    if (stat("hello.txt", &st) == -1) {
        perror("stat");
        return 1;
    }

    printf("Размер: %ld байт\n", (long)st.st_size);
    printf("Inode: %ld\n", (long)st.st_ino);
    printf("Ссылка: %ld\n", (long)st.st_nlink);
    return 0;
}
```

Что показывает: структура struct stat хранит **метаданные файла**: размер, inode, число ссылок.

Пример 14. Тип файла через st_mode

```
#include <stdio.h>
#include <sys/stat.h>

int main(void) {
    struct stat st;
    if (stat("hello.txt", &st) == -1) return 1;

    printf("Тип: ");
    if (S_ISREG(st.st_mode))    printf("обычный файл\n");
    else if (S_ISDIR(st.st_mode)) printf("каталог\n");
    else if (S_ISLNK(st.st_mode)) printf("символьная ссылка\n");
    else                        printf("другое\n");

    return 0;
}
```

Что показывает: поле st_mode хранит **тип файла**. Проверяется макросами S_ISREG, S_ISDIR, S_ISLNK.

Пример 15. Права доступа через st_mode

```
#include <stdio.h>
#include <sys/stat.h>

int main(void) {
    struct stat st;
    if (stat("hello.txt", &st) == -1) return 1;

    printf("Права: ");
    printf("%c", (st.st_mode & S_IRUSR) ? 'r' : '-');
    printf("%c", (st.st_mode & S_IWUSR) ? 'w' : '-');
    printf("%c", (st.st_mode & S_IXUSR) ? 'x' : '-');
    printf("%c", (st.st_mode & S_IRGRP) ? 'r' : '-');
    printf("%c", (st.st_mode & S_IWGRP) ? 'w' : '-');
    printf("%c", (st.st_mode & S_IXGRP) ? 'x' : '-');
    printf("%c", (st.st_mode & S_IROTH) ? 'r' : '-');
    printf("%c", (st.st_mode & S_IWOTH) ? 'w' : '-');
    printf("%c\n", (st.st_mode & S_IXOTH) ? 'x' : '-');

    return 0;
}
```

Что показывает: права доступа **упакованы** в st_mode — три группы по три бита (владелец, группа, остальные).

Пример 16. Временные метки файла

```
#include <stdio.h>
#include <sys/stat.h>
#include <time.h>

int main(void) {
    struct stat st;
    if (stat("hello.txt", &st) == -1) return 1;

    printf("Последний доступ: %s", ctime(&st.st_atime));
    printf("Последнее изменение: %s", ctime(&st.st_mtime));
    printf("Изменение inode:  %s", ctime(&st.st_ctime));
    return 0;
}
```

Что показывает: struct stat хранит **три времени**:

st_atime — доступ,

st_mtime — изменение содержимого,

st_ctime — изменение inode.

Пример 17. fstat — метаданные по дескриптору

```
#include <fcntl.h>
#include <sys/stat.h>
#include <unistd.h>
#include <stdio.h>

int main(void) {
    int fd = open("hello.txt", O_RDONLY);
    if (fd == -1) return 1;

    struct stat st;
    if (fstat(fd, &st) == -1) return 1;

    printf("Размер: %ld\n", (long)st.st_size);
    close(fd);
    return 0;
}
```

Что показывает: fstat работает **по дескриптору**, а не по имени файла. Полезно, когда файл уже открыт.

Часть 4. Специальные структуры (примеры 18–20)



Пример 18. Структура DIR для каталогов

```
#include <stdio.h>
#include <dirent.h>
```

```
int main(void) {
    DIR *dir = opendir(".");
    if (!dir) return 1;

    struct dirent *entry;
    while ((entry = readdir(dir)) != NULL)
        printf("%s\n", entry->d_name);

    closedir(dir);
    return 0;
}
```

Что показывает: для каталогов используется **своя структура** — DIR и struct dirent. Поле d_name хранит имя файла.

Пример 19. Структура passwd — информация о пользователе

```
#include <stdio.h>
#include <pwd.h>

int main(void) {
    struct passwd *pw = getpwuid(getuid());
    if (!pw) return 1;

    printf("Пользователь: %s\n", pw->pw_name);
    printf("UID: %d\n", pw->pw_uid);
    printf("Домашний каталог: %s\n", pw->pw_dir);
    return 0;
}
```

Что показывает: в системе есть **другие структуры** для информации о пользователях, группах, устройствах. struct passwd — одна из них.

Пример 20. Сводная информация о файле

```
#include <stdio.h>
#include <sys/stat.h>

int main(void) {
    struct stat st;
    if (stat("hello.txt", &st) == -1) return 1;

    printf("=== Информация о файле ===\n");
    printf("Размер:    %ld байт\n", (long)st.st_size);
    printf("Inode:    %ld\n", (long)st.st_ino);
    printf("Устройство: %ld\n", (long)st.st_dev);
    printf("Ссылка:    %ld\n", (long)st.st_nlink);
    printf("UID:      %d\n", st.st_uid);
    printf("GID:      %d\n", st.st_gid);
    printf("Блоков:    %ld\n", (long)st.st_blocks);
    printf("Размер блока: %ld\n", (long)st.st_blksize);

    return 0;
}
```

Что показывает: все основные поля struct stat в одном примере — **полный «паспорт» файла.**

Сводная таблица: структуры и их назначение

Структура	Где определена	Что хранит
<code>FILE</code>	<code><stdio.h></code>	Буфер, дескриптор, позиция, флаги
<code>struct stat</code>	<code><sys/stat.h></code>	Метаданные: размер, тип, права, время
<code>int fd</code>	—	Файловый дескриптор
<code>DIR</code>	<code><dirent.h></code>	Информация о каталоге
<code>struct dirent</code>	<code><dirent.h></code>	Имя файла, inode
<code>struct passwd</code>	<code><pwd.h></code>	Информация о пользователе
<code>struct tm</code>	<code><time.h></code>	Разложенное время

Что важно запомнить

FILE — непрозрачная структура. Её поля нельзя читать напрямую — только через функции.

struct stat — основной источник **метаданных** файла.

Дескриптор (int fd) — низкоуровневый «пропуск» к файлу.

fileno связывает FILE и дескриптор.

fdopen делает обратное — оборачивает дескриптор в FILE.

Итог

20 примеров покрывают:

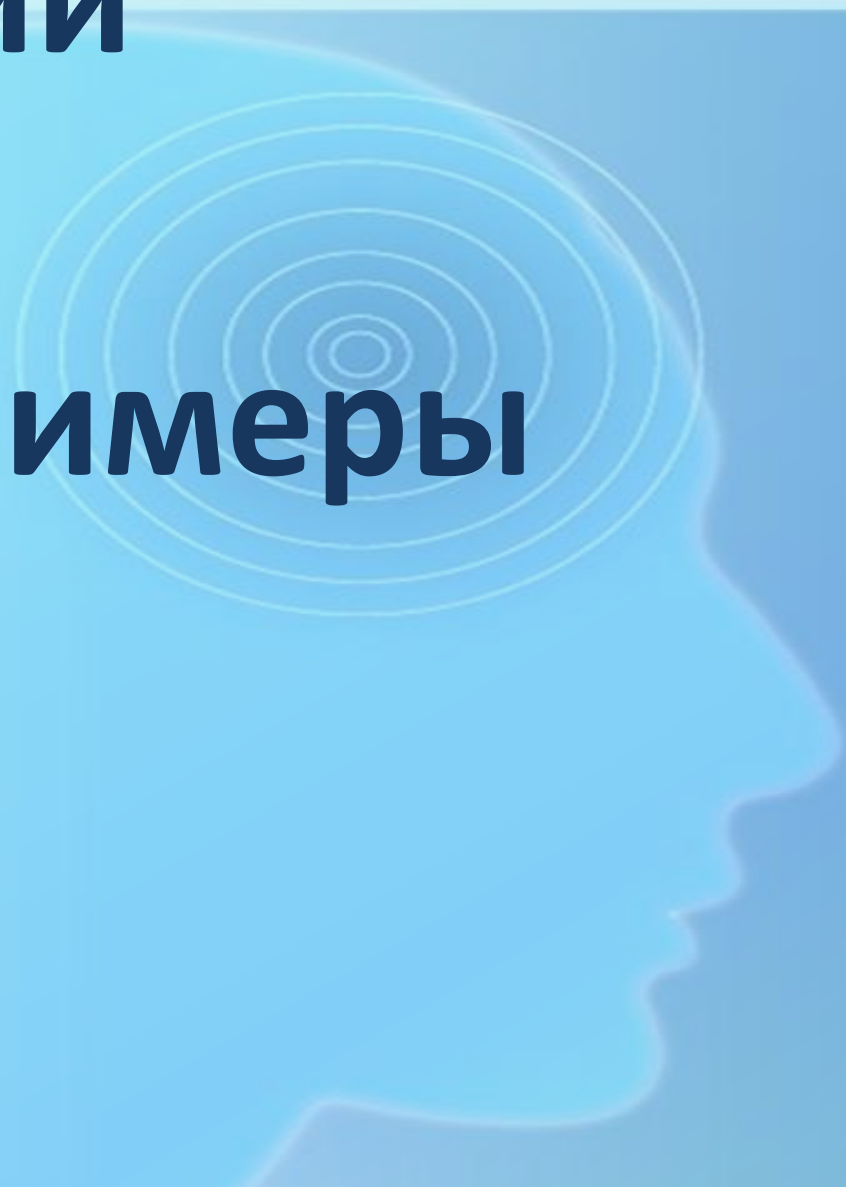
- **FILE *** — высокоуровневая структура,
- **файловые дескрипторы** — низкоуровневый доступ,
- **struct stat** — метаданные,
- **DIR / dirent** — каталоги,
- **passwd** — пользователи.

Эти примеры можно использовать как шаблоны.

Функции и системные вызовы для работы с файлами»



Часть 1. Функции стандартной библиотеки (примеры 1–20)



Пример 1. Чтение файла посимвольно (fgetc)

```
#include <stdio.h>
```

```
int main(void) {  
    FILE *fp = fopen("hello.txt", "r");  
    if (!fp) return 1;  
  
    int c;  
    while ((c = fgetc(fp)) != EOF)  
        printf("%c", c);  
  
    fclose(fp);  
    return 0;  
}
```

Что делает: читает файл **по одному символу** до конца (EOF).



Пример 2. Чтение файла построчно (fgets)

```
#include <stdio.h>
```

```
int main(void) {  
    FILE *fp = fopen("hello.txt", "r");  
    if (!fp) return 1;  
  
    char buf[256];  
    while (fgets(buf, sizeof(buf), fp))  
        printf("%s", buf);  
  
    fclose(fp);  
    return 0;  
}
```

Что делает: читает файл **по строкам** (до 255 символов за раз).

Пример 3. Чтение с проверкой на ошибку (perror)

- `#include <stdio.h>`
- `int main(void) {`
- `FILE *fp = fopen("hello.txt", "r");`
- `if (fp == NULL) {`
- `perror("fopen");`
- `return 1;`
- `}`
- `fclose(fp);`
- `return 0;`
- `}`
- **Что делает:** если файл не открылся — выводит **сообщение об ошибке** через `perror`.

Пример 4. Подсчёт символов

```
#include <stdio.h>

int main(void) {
    FILE *fp = fopen("hello.txt", "r");
    if (!fp) return 1;

    int c, count = 0;
    while ((c = fgetc(fp)) != EOF) count++;

    printf("Символов: %d\n", count);
    fclose(fp);
    return 0;
}
```

Что делает: считает количество символов в файле.



Пример 5. Подсчёт строк

```
#include <stdio.h>

int main(void) {
    FILE *fp = fopen("hello.txt", "r");
    if (!fp) return 1;

    int c, lines = 0;
    while ((c = fgetc(fp)) != EOF)
        if (c == '\n') lines++;

    printf("Строк: %d\n", lines);
    fclose(fp);
    return 0;
}
```

Что делает: считает количество строк (по символу \n).

Пример 6. Запись строки (fputs)

```
#include <stdio.h>
```

```
int main(void) {  
    FILE *fp = fopen("out.txt", "w");  
    if (!fp) return 1;  
  
    fputs("Привет, мир!\n", fp);  
  
    fclose(fp);  
    return 0;  
}
```

Что делает: записывает строку в файл. Режим "w" **перезаписывает** файл.

Пример 7. Запись посимвольно (fputc)

```
#include <stdio.h>
```

```
int main(void) {  
    FILE *fp = fopen("out.txt", "w");  
    if (!fp) return 1;  
  
    const char *s = "Hello";  
    for (int i = 0; s[i]; i++)  
        fputc(s[i], fp);  
  
    fclose(fp);  
    return 0;  
}
```

Что делает: записывает каждый символ отдельно через fputc.

Пример 8. Форматированная запись (fprintf)

```
#include <stdio.h>
```

```
int main(void) {  
    FILE *fp = fopen("out.txt", "w");  
    if (!fp) return 1;  
  
    fprintf(fp, "Число: %d\n", 42);  
    fprintf(fp, "Строка: %s\n", "test");  
  
    fclose(fp);  
    return 0;  
}
```

Что делает: записывает данные с форматированием, как printf, но в файл.

Пример 9. Дозапись в конец ("a")

```
#include <stdio.h>
```

```
int main(void) {  
    FILE *fp = fopen("out.txt", "a");  
    if (!fp) return 1;  
  
    fputs("Новая строка\n", fp);  
  
    fclose(fp);  
    return 0;  
}
```

Что делает: режим "a" **добавляет** данные в конец файла, не стирая старое.

Пример 10. Запись массива чисел

```
#include <stdio.h>

int main(void) {
    FILE *fp = fopen("numbers.txt", "w");
    if (!fp) return 1;

    int arr[5] = {1, 2, 3, 4, 5};
    for (int i = 0; i < 5; i++)
        fprintf(fp, "%d\n", arr[i]);

    fclose(fp);
    return 0;
}
```

Что делает: записывает массив чисел в файл — каждое на своей строке.

Пример 11. Копирование файла

```
#include <stdio.h>

int main(void) {
    FILE *in = fopen("in.txt", "r");
    FILE *out = fopen("out.txt", "w");
    if (!in || !out) return 1;

    int c;
    while ((c = fgetc(in)) != EOF)
        fputc(c, out);

    fclose(in);
    fclose(out);
    return 0;
}
```

Что делает: копирует содержимое файла in.txt в файл out.txt — посимвольно

Подключение библиотеки

```
#include <stdio.h>
```

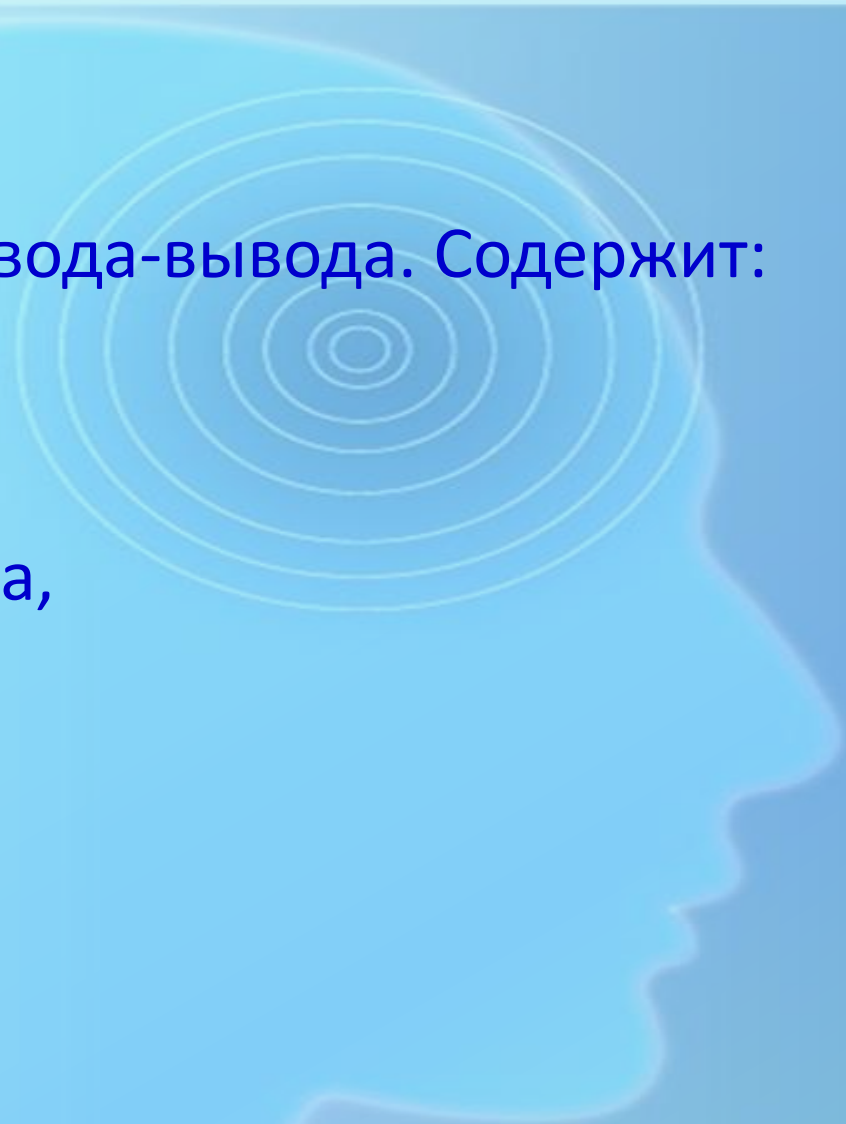
stdio.h — стандартная библиотека ввода-вывода. Содержит:

FILE — тип для работы с файлами,

fopen, fclose — открытие/заккрытие,

fgetc, fputc — чтение/запись символа,

EOF — признак конца файла.



Открытие исходного файла

- `FILE *in = fopen("in.txt", "r");`
- `fopen("in.txt", "r")` — открывает файл `in.txt` **только для чтения**.
- Режим `"r"`: файл **должен существовать**, иначе `fopen` вернёт `NULL`.
- `FILE *in` — указатель на структуру `FILE` — «дескриптор» файла.

Открытие файла для записи

```
FILE *out = fopen("out.txt", "w");
```

Режим "w": файл открывается **для записи**.

Если файла **нет** — он **создаётся**.

Если файл **есть** — его
содержимое **стирается** (перезаписывается).

Проверка на ошибки

```
if (!in || !out) return 1;
```

!in — истина, если in == NULL (файл не открылся).

!out — то же для out.

Если **хотя бы один** файл не открылся — выходим с кодом 1 (ошибка).

Важно: проверять нужно **оба** указателя.

Чтение и запись посимвольно

```
int c;  
while ((c = fgetc(in)) != EOF)  
    fputc(c, out);
```

Как это работает:

`fgetc(in)` — читает **один символ** из `in.txt`.

Результат сохраняется в `c` (тип `int`, а не `char` — важно!).

Если `c == EOF` — цикл завершается (конец файла).

`fputc(c, out)` — записывает символ `c` в `out.txt`.

Повторяется, пока не достигнут конец файла.

Почему int, а не char?

`fgetc` возвращает `int` — чтобы можно было отличить **конец файла (EOF)** от **любого символа**.

EOF — это **-1** (отрицательное число). Если бы `c` был `char`, то символ с кодом 255 мог бы «слиться» с EOF.

Заккрытие файлов

`fclose(in);`

`fclose(out);`

Сбрасывает буфер — все незаписанные данные сохраняются на диск.

Освобождает ресурсы — дескрипторы возвращаются системе.

Обязательно вызывать `fclose` — иначе данные могут не сохраниться.

Завершение программы

```
return 0;
```

Возврат 0 — успешное завершение.

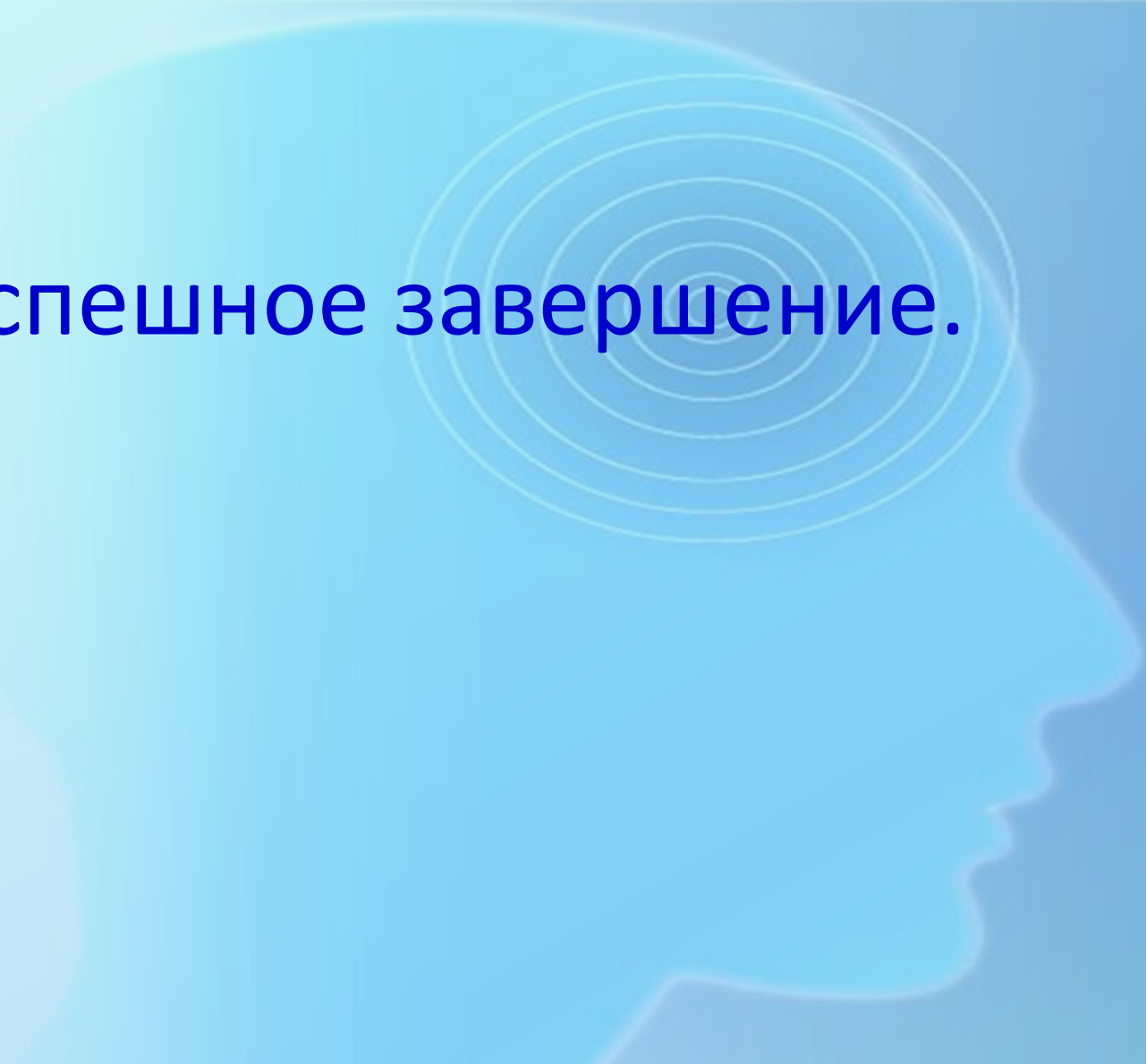
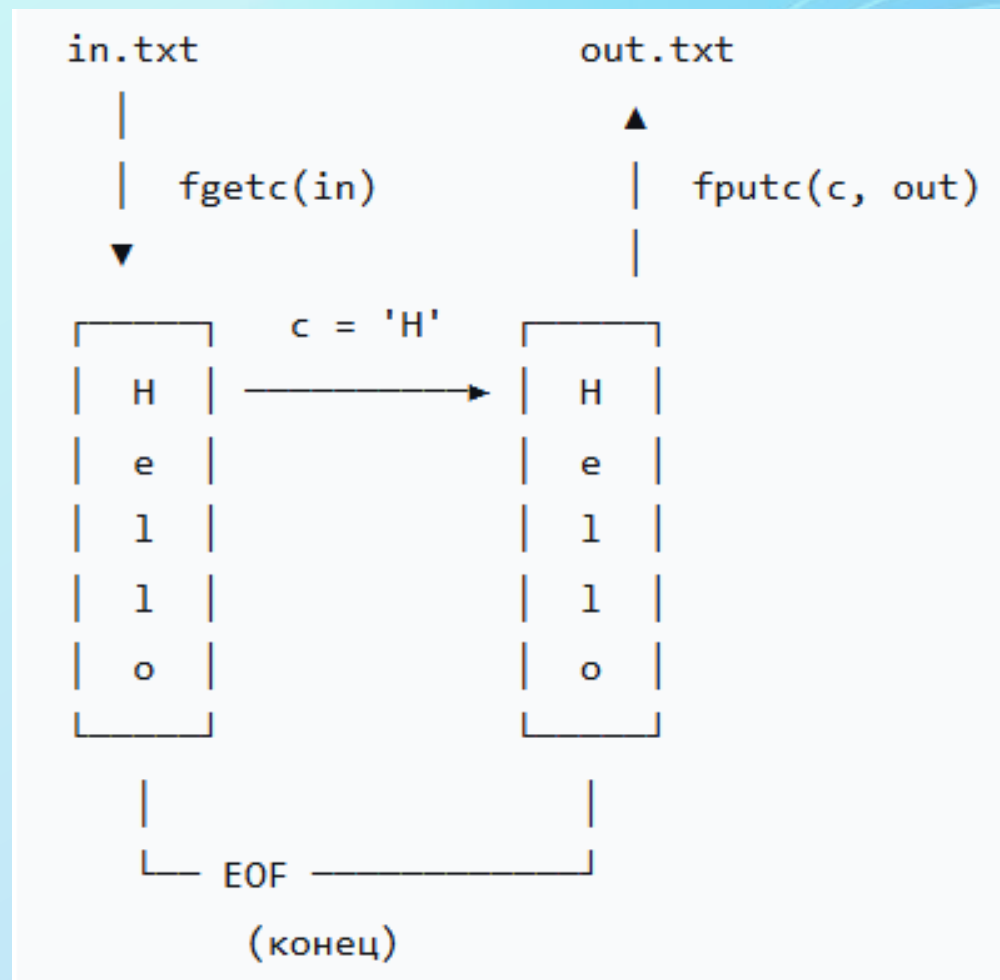


Схема работы



Пошагово:

c = 'H' → fputc('H', out)

c = 'e' → fputc('e', out)

c = 'l' → fputc('l', out)

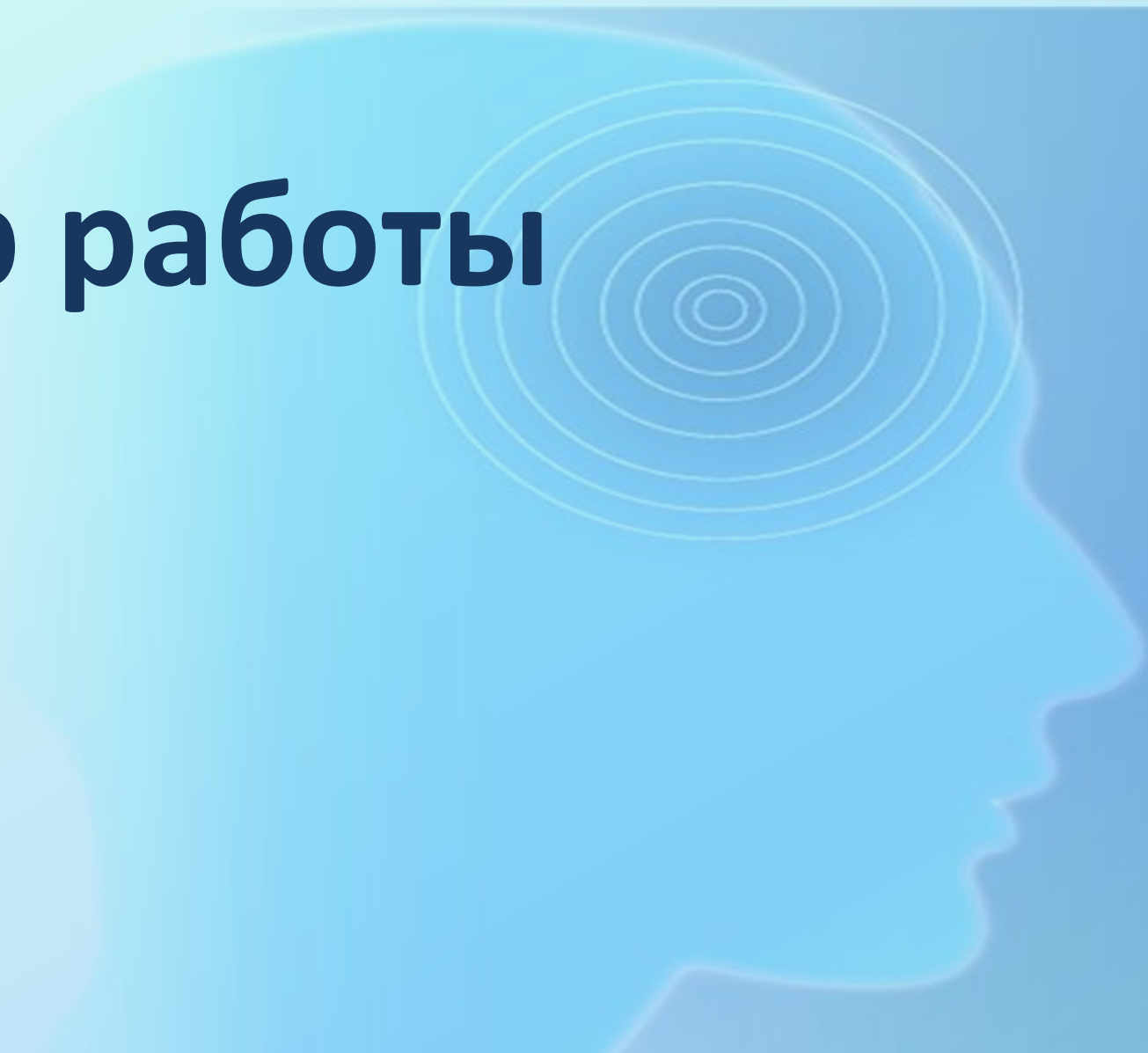
c = 'l' → fputc('l', out)

c = 'o' → fputc('o', out)

c = EOF → выход из цикла



Пример работы



До запуска:

in.txt:

text

Hello, world!

out.txt (пустой или не существует).



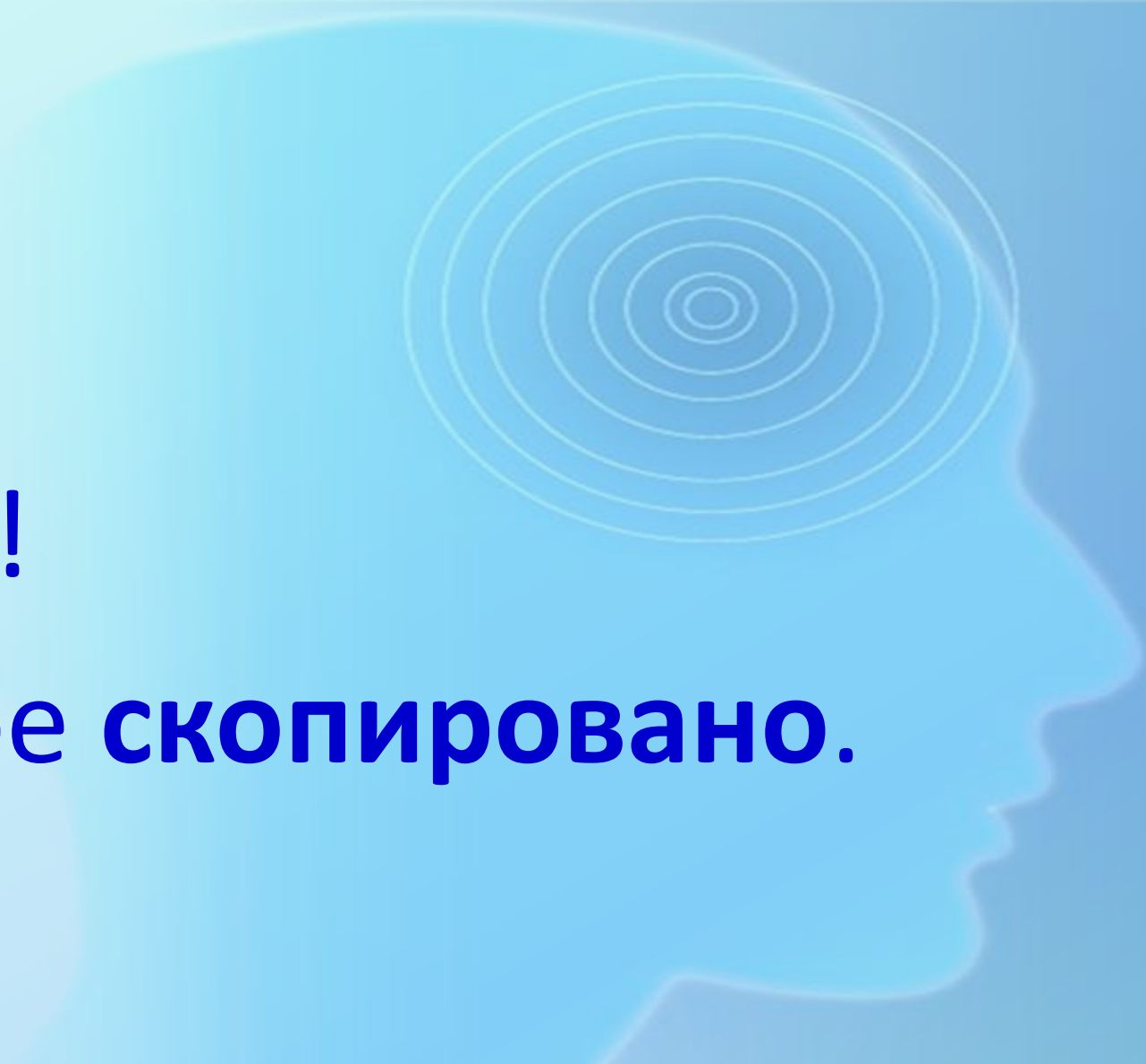
После запуска:

out.txt:

text

Hello, world!

Содержимое скопировано.



Что важно запомнить



Элемент	Что делает
<code>fopen(name, "r")</code>	Открыть для чтения (файл должен существовать)
<code>fopen(name, "w")</code>	Открыть для записи (создать или перезаписать)
<code>fgetc(fp)</code>	Читать символ, вернуть <code>int</code>
<code>fputc(c, fp)</code>	Записать символ
<code>EOF</code>	Признак конца файла (-1)
<code>fclose(fp)</code>	Заккрыть, сбросить буфер
<code>if (!in !out)</code>	Проверить, что оба файла открыты

Пример 12. Перевод в верхний регистр

```
#include <stdio.h>
#include <ctype.h>

int main(void) {
    FILE *fp = fopen("in.txt", "r");
    if (!fp) return 1;

    int c;
    while ((c = fgetc(fp)) != EOF)
        putchar(toupper(c));

    fclose(fp);
    return 0;
}
```

Что делает: читает файл и выводит в верхнем регистре.



Обязательные заголовки:

`<stdio.h>` —
для `fopen`, `fgetc`, `putchar`, `fclose`.
`<ctype.h>` — для `toupper`.

`c = fgetc(fp)` — чтение символа

`fgetc(fp)` — читает **один символ** из файла, на который указывает `fp`.
Возвращает `int` — код символа или EOF (-1), если достигнут конец файла.
Результат сохраняется в переменной `c` (тип `int`).

`(c = fgetc(fp)) != EOF` — проверка на конец файла

Скобки вокруг `c = fgetc(fp)` **обязательны** — иначе приоритет операций нарушится.

Сначала **читаем** символ, потом **сравниваем** с EOF.

Если символ не EOF — условие **истинно**, тело цикла выполняется.

Почему `c` — `int`, а не `char`?

EOF = -1 (отрицательное число).

Если `c` был бы `char`, символ с кодом 255 мог бы «слиться» с EOF.

Поэтому `fgetc` возвращает `int`.

tolower(c) — перевод в верхний регистр

tolower(c) — функция из <ctype.h>.

Если c — **строчная буква** ('a'..'z') —
возвращает **заглавную** ('A'..'Z').

Если c — **не буква** (цифра, пробел, знак) —
возвращает **без изменений**.

`putchar(...)` — вывод символа

`putchar(x)` — выводит
символ `x` в **`stdout`** (на экран).
Эквивалент `printf("%c", x)`.

Пример 13. Удаление пустых строк

```
#include <stdio.h>
#include <string.h>

int main(void) {
    FILE *fp = fopen("in.txt", "r");
    if (!fp) return 1;

    char buf[256];
    while (fgets(buf, sizeof(buf), fp))
        if (strcmp(buf, "\n") != 0)
            printf("%s", buf);

    fclose(fp);
    return 0;
}
```

Что делает: пропускает пустые строки.



• Что такое пустая строка?

Строка, содержащая **только символ перевода строки \n**.

Например, в файле:

text

Первая строка

Третья строка

Вторая строка — **пустая**, содержит только \n.

Обязательные заголовки:

`<stdio.h>` —
для `fopen`, `fgets`, `printf`, `fclose`.

`<string.h>` — для `strcmp`.

`char buf[256]` — буфер для строки

Массив на **256 символов**.

Сюда `fgets` будет записывать **очередную строку**.

Максимальная длина строки — **255 символов + `\0`** (нулевой символ-терминатор).

`fgets(buf, sizeof(buf), fp)` — чтение строки

`fgets(buf, sizeof(buf), fp)`

Аргументы:

`buf` — куда читать.

`sizeof(buf) = 256` — **максимальное количество** символов для чтения.

`fp` — откуда читать (указатель на файл).

Возвращает:

`buf` — если строка прочитана успешно.

`NULL` — если **конец файла** или **ошибка**.

Что читает:

Читает **до 255 символов** или **до символа \n** (включительно).

Добавляет \0 в конец.

Если строка длиннее 255 символов — она читается **частями**.

`while (fgets(...))` — цикл по строкам

```
while (fgets(buf, sizeof(buf), fp))
```

Пока `fgets` возвращает **не NULL** — цикл продолжается.

Когда достигнут конец файла — `fgets` вернёт `NULL` → цикл завершится.

Почему без `!= NULL`?

В C **любое ненулевое** значение — истина.

`NULL = 0` — ложь.

Поэтому `while (fgets(...))` — то же, что `while (fgets(...) != NULL)`.

`strcmp(buf, "\n") != 0` — проверка на пустую строку

`strcmp(buf, "\n") != 0`

`strcmp(s1, s2)` — сравнивает две строки:

Возвращает **0**, если строки **равны**.

Возвращает **< 0** или **> 0**, если строки **различаются**.

Что проверяем:

Если `buf` **равен** `"\n"` → `strcmp` вернёт **0** → условие `!= 0` **ложно** → строка **пропускается**.

Если `buf` **не равен** `"\n"` → `strcmp` вернёт **не 0** → условие **истинно** → строка **выводится**.

`printf("%s", buf)` — вывод строки

`printf("%s", buf)` — выводит строку из `buf`.

Без `\n` в конце — потому что `\n` **уже** есть в `buf` (от `fgets`).

Почему не `puts`?

`puts(buf)` добавил бы **ещё один `\n`** → было бы **два перевода строки**.

Схема работы

Файл		fgets	strcmp(buf, "\n")	printf
"Первая\n"	→	buf="Первая\n"	!= 0	→ истина → вывод
"\n"	→	buf="\n"	== 0	→ ложь → пропуск
"Третья\n"	→	buf="Третья\n"	!= 0	→ истина → вывод
"\n"	→	buf="\n"	== 0	→ ложь → пропуск
"\n"	→	buf="\n"	== 0	→ ложь → пропуск
"Шестая\n"	→	buf="Шестая\n"	!= 0	→ истина → вывод
NULL	→	выход из цикла		

Что важно запомнить

Элемент

Что делает

```
char buf[256]
```

Буфер на 256 символов

```
fgets(buf, size, fp)
```

Читает строку (до `\n` или `size-1`)

```
sizeof(buf)
```

Размер буфера (256)

```
strcmp(s1, s2)
```

Сравнивает строки (0 — равны)

```
"\n"
```

Строка из одного символа перевода строки

```
printf("%s", buf)
```

Выводит строку **как есть**

Элемент	Что делает
<code>char buf[256]</code>	Буфер на 256 символов
<code>fgets(buf, size, fp)</code>	Читает строку (до <code>\n</code> или <code>size-1</code>)
<code>sizeof(buf)</code>	Размер буфера (256)
<code>strcmp(s1, s2)</code>	Сравнивает строки (0 — равны)
<code>"\n"</code>	Строка из одного символа перевода строки
<code>printf("%s", buf)</code>	Выводит строку как есть

Пример 14. Поиск слова в файле

```
#include <stdio.h>
#include <string.h>

int main(void) {
    FILE *fp = fopen("in.txt", "r");
    if (!fp) return 1;

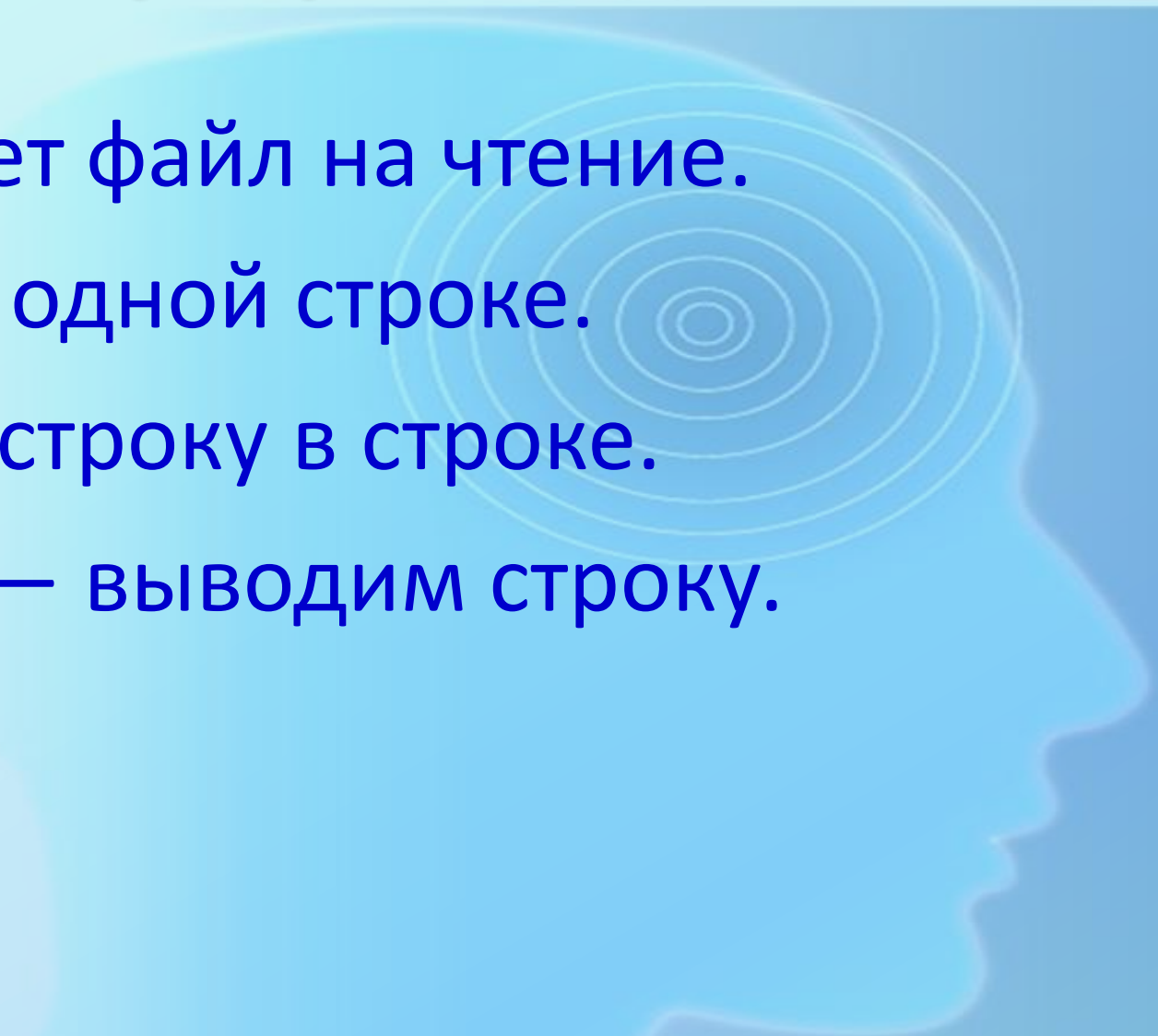
    char buf[256];
    int line = 0;
    while (fgets(buf, sizeof(buf), fp)) {
        line++;
        if (strstr(buf, "hello"))
            printf("Строка %d: %s", line, buf);
    }

    fclose(fp);
    return 0;
}
```

Что делает: ищет **слово** в файле и выводит номер строки.



Что тут происходит:

- fopen открывает файл на чтение.
 - fgets читает по одной строке.
 - strstr ищет подстроку в строке.
 - Если найдено — выводим строку.
- 

Пример 15. Слияние двух файлов

```
#include <stdio.h>
```

```
int main(void) {  
    FILE *f1 = fopen("a.txt", "r");  
    FILE *f2 = fopen("b.txt", "r");  
    FILE *out = fopen("c.txt", "w");  
    if (!f1 || !f2 || !out) return 1;  
  
    int c;  
    while ((c = fgetc(f1)) != EOF) fputc(c, out);  
    while ((c = fgetc(f2)) != EOF) fputc(c, out);  
  
    fclose(f1);  
    fclose(f2);  
    fclose(out);  
    return 0;  
}
```

Что делает: сливает два файла в один.



Пример 16. Размер файла через fseek + ftell

```
#include <stdio.h>

int main(void) {
    FILE *fp = fopen("hello.txt", "r");
    if (!fp) return 1;

    fseek(fp, 0, SEEK_END);
    long size = ftell(fp);
    printf("Размер: %ld байт\n", size);

    fclose(fp);
    return 0;
}
```

Что делает: перемещает указатель в конец и получает **размер** файла.

Пример 17. Чтение с середины файла

```
#include <stdio.h>

int main(void) {
    FILE *fp = fopen("hello.txt", "r");
    if (!fp) return 1;

    fseek(fp, 5, SEEK_SET);

    int c;
    while ((c = fgetc(fp)) != EOF)
        putchar(c);

    fclose(fp);
    return 0;
}
```

Что делает: начинает чтение с 6-го байта файла.



Пример 18. Перемотка в начало (rewind)

```
#include <stdio.h>

int main(void) {
    FILE *fp = fopen("hello.txt", "r");
    if (!fp) return 1;

    int c;
    while ((c = fgetc(fp)) != EOF) putchar(c);

    rewind(fp);

    printf("\n--- Второй проход ---\n");
    while ((c = fgetc(fp)) != EOF) putchar(c);

    fclose(fp);
    return 0;
}
```

Что делает: читает файл **дважды** — использует `rewind`



Пример 19. Метаданные через stat

```
#include <stdio.h>
#include <sys/stat.h>

int main(void) {
    struct stat st;
    if (stat("hello.txt", &st) == -1) {
        perror("stat");
        return 1;
    }

    printf("Размер: %ld байт\n", (long)st.st_size);
    printf("Тип: %s\n",
        S_ISREG(st.st_mode) ? "файл" :
        S_ISDIR(st.st_mode) ? "каталог" : "другое");

    return 0;
}
```

Что делает: получает **метаданные** файла через stat.



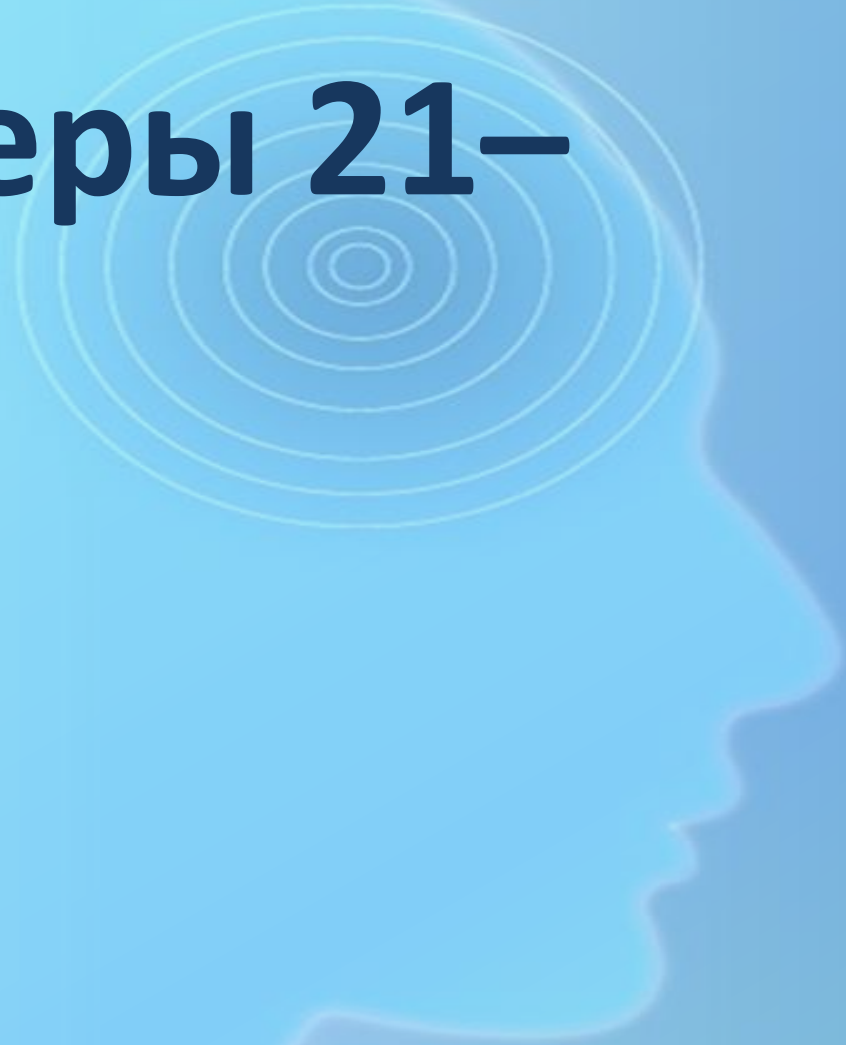
Пример 20. Проверка существования файла

```
#include <stdio.h>
```

```
int main(void) {  
    FILE *fp = fopen("hello.txt", "r");  
    if (fp) {  
        printf("Файл существует\n");  
        fclose(fp);  
    } else {  
        printf("Файл не найден\n");  
    }  
    return 0;  
}
```

Что делает: проверяет, **существует ли файл** — через попытку открытия.

Часть 2. Системные вызовы (примеры 21– 25)



Пример 21. Открытие файла через open

```
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>

int main(void) {
    int fd = open("hello.txt", O_RDONLY);
    if (fd == -1) {
        perror("open");
        return 1;
    }
    printf("Дескриптор: %d\n", fd);
    close(fd);
    return 0;
}
```

Что делает: открывает файл **системным вызовом** open. Возвращает **файловый дескриптор** (int).

Разбор флагов:

O_RDONLY — только чтение.

O_WRONLY — только запись.

O_RDWR — чтение и запись.

O_CREAT — создать, если нет.

O_TRUNC — обрезать до нуля.

Пример 22. Чтение через read

```
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>

int main(void) {
    int fd = open("hello.txt", O_RDONLY);
    if (fd == -1) return 1;

    char buf[256];
    ssize_t n;
    while ((n = read(fd, buf, sizeof(buf))) > 0)
        write(1, buf, n); // вывод в stdout

    close(fd);
    return 0;
}
```



Что делает: читает файл блоками через системный вызов `read`.

Разбор:

`read(fd, buf, count)` — читает до `count` байт.

Возвращает число прочитанных байт, 0 — конец, -1 — ошибка.

`write(1, buf, n)` — вывод в **`stdout`** (дескриптор 1).

Пример 23. Запись через write

```
#include <fcntl.h>
#include <unistd.h>

int main(void) {
    int fd = open("out.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd == -1) return 1;

    const char *msg = "Hello\n";
    write(fd, msg, 6);

    close(fd);
    return 0;
}
```

Что делает: записывает строку в файл через write.

Разбор:

O_WRONLY | O_CREAT | O_TRUNC — открыть для записи, создать, обрезать.

0644 — права доступа: владелец — чтение/запись, остальные — чтение.

write(fd, buf, count) — записывает до count байт.

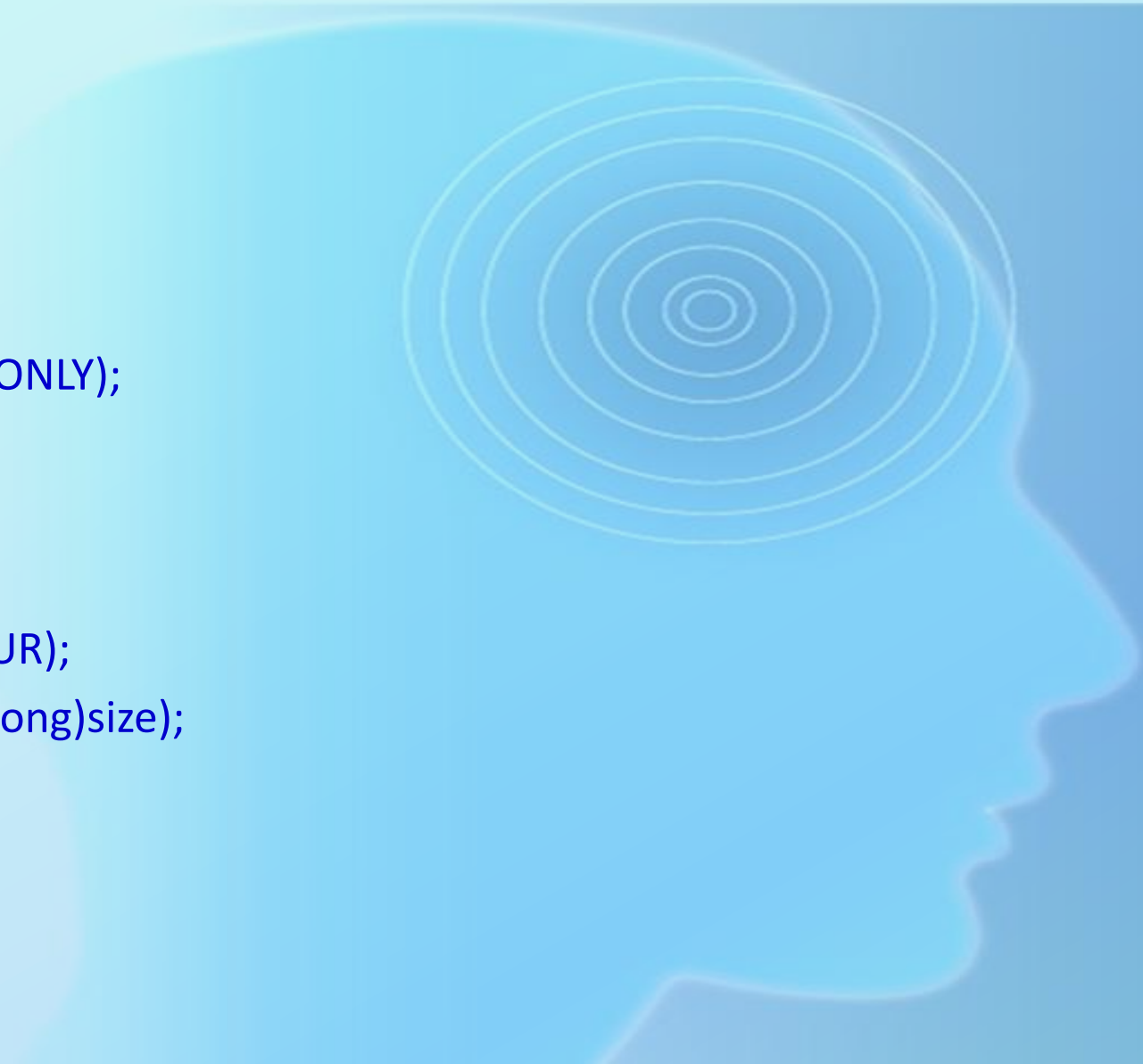
Пример 24. Позиционирование через lseek

```
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>

int main(void) {
    int fd = open("hello.txt", O_RDONLY);
    if (fd == -1) return 1;

    lseek(fd, 0, SEEK_END);
    off_t size = lseek(fd, 0, SEEK_CUR);
    printf("Размер: %ld байт\n", (long)size);

    close(fd);
    return 0;
}
```



Что делает: перемещает позицию через lseek и получает размер файла.

Разбор:

lseek(fd, offset, origin) — перемещает указатель.

SEEK_SET — от начала, SEEK_CUR — от текущей, SEEK_END — от конца.

Возвращает **новую позицию**.

Пример 25. Метаданные через fstat

```
#include <fcntl.h>
#include <sys/stat.h>
#include <unistd.h>
#include <stdio.h>

int main(void) {
    int fd = open("hello.txt", O_RDONLY);
    if (fd == -1) return 1;

    struct stat st;
    if (fstat(fd, &st) == -1) return 1;

    printf("Размер: %ld\n", (long)st.st_size);
    printf("Inode: %ld\n", (long)st.st_ino);
    printf("Тип: %s\n",
        S_ISREG(st.st_mode) ? "файл" :
        S_ISDIR(st.st_mode) ? "каталог" : "другое");

    close(fd);
    return 0;
}
```



Что делает: получает метаданные по открытому дескриптору через fstat.

Отличие от stat:

stat(path, &st) — по имени файла.

fstat(fd, &st) — по открытому дескриптору.

lstat(path, &st) — не
разыменовывает **символические ссылки**.

Сводная таблица: функции vs системные ВЫЗОВЫ

Задача	Функция (FILE *)	Системный вызов (int fd)
Открыть	fopen	open
Закрыть	fclose	close
Читать символ	fgetc	read (по 1 байту)
Читать блок	fread	read
Читать строку	fgets	—
Писать символ	fputc	write (по 1 байту)
Писать блок	fwrite	write
Писать строку	fputs	—
Форматировать	fprintf	—
Позиция	fseek , ftell	lseek

Позиция	<code>fseek</code> , <code>ftell</code>	<code>lseek</code>
Перемотка	<code>rewind</code>	<code>lseek(fd, 0, SEEK_SET)</code>
Метаданные	<code>stat</code> , <code>fstat</code>	<code>stat</code> , <code>fstat</code>
Буферизация	Есть	Нет

Режимы открытия — сравнение

for open

Режим	Значение
"r"	Чтение
"w"	Запись (перезапись)
"a"	Дозапись
"r+"	Чтение и запись
"w+"	Чтение и запись (перезапись)
"a+"	Чтение и дозапись
"rb" , "wb"	Бинарные режимы

open

Флаг	Значение
<code>O_RDONLY</code>	Только чтение
<code>O_WRONLY</code>	Только запись
<code>O_RDWR</code>	Чтение и запись
<code>O_CREAT</code>	Создать, если нет
<code>O_TRUNC</code>	Обрезать до нуля
<code>O_APPEND</code>	Писать в конец
<code>O_EXCL</code>	Ошибка, если файл есть

Что важно запомнить

Структуры хранения информации о файле

- **FILE *** — высокоуровневая структура (буфер, дескриптор, позиция, флаги).
- **int fd** — файловый дескриптор (низкоуровневый).
- **struct stat** — метаданные файла (размер, тип, права, время).

Функции

- Работают с FILE *.
- **Буферизованы** — быстрее при мелких операциях.
- **Платформо-независимы** — стандарт C.

Системные вызовы

- Работают с `int fd`.
- **Не буферизованы** — каждое обращение к диску.
- **POSIX** — Unix/Linux.
- Прямой доступ к ядру ОС.